

PROJECT DESIGN & REQUIREMENT REPORT

Food Delivery Application (FoodFlow)

Process Model: Kanban (Continuous Workflow with WIP Limits)

Project Group:	Group 11 (Section A)
Assigned Project:	Food Delivery Application
Process Model:	Kanban Agile Framework
Technology Stack:	Python 3.14, Flask, SQLite3, HTML5/Tailwind CSS
Target Platform:	Web & Mobile-Responsive Browser Application
Submission Date:	September 2026

1. Project Title & Overview

FoodFlow is an on-demand multi-tenant food ordering and delivery management system developed for RUET campus and surrounding urban areas. The system seamlessly connects hungry customers, partner restaurant kitchens, delivery drivers, and platform administrators through a responsive web application and unified Kanban workflow pipeline.

2. Problem Statement

Traditional food ordering systems in local university ecosystems suffer from fragmented telephone orders, lack of order progress transparency, kitchen overcrowding during peak dining hours, and inefficient dispatching of delivery couriers. Without strict Work-In-Progress (WIP) controls, restaurant kitchens experience severe preparation bottlenecks, resulting in delayed orders, cold food, and customer dissatisfaction.

3. Project Objectives

- Deliver a functional Minimum Viable Product (MVP) enabling customer ordering, restaurant menu management, and courier dispatching.
- Enforce Kanban process principles both in the software development lifecycle and the real-time order fulfillment workflow.
- Eliminate kitchen bottlenecks and decrease order fulfillment lead times through explicit Work-In-Progress (WIP) limits.
- Provide real-time transparency for customers to track order progress across every milestone from preparation to doorstep delivery.

4. Stakeholder Analysis

Stakeholder	Role Description	Key Needs & Expectations
Customer / Foodie	End-users ordering meals for personal consumption	Fast menu search, seamless cart checkout, accurate ETA, live tracking, fair pricing
Restaurant Owner	Kitchen managers offering culinary dishes	Instant order notifications, menu item CRUD, inventory availability toggle, kitchen WIP limit controls
Delivery Rider	Logistics personnel transporting meals	Instant job pickup alerts, pickup/drop-off directions, delivery fee earnings summary
System Admin	Platform operations & regulatory manager	System-wide analytics, revenue reporting, dispute resolution, user governance, service health

5. Functional Requirements (FR)

ID	Requirement Name	Description & Acceptance Criteria
FR1	User Registration & Auth	Support secure registration and login with bcrypt password hashing and Role-Based Access Control (Customer, Owner, Rider, Admin).
FR2	Restaurant & Menu Catalog	Display active restaurants with cuisine filtering (Italian, Japanese, Fast Food) and live text search.
FR3	Menu Management (CRUD)	Restaurant owners can add dishes, update prices/descriptions, upload image URLs, and toggle item availability.
FR4	Shopping Cart & Checkout	Customers can add items, modify quantities, apply coupon discounts (e.g. RUET10), and select payment methods.
FR5	Order Lifecycle & State Transitions	Order transitions sequentially through Kanban states: Placed -> Preparing -> Ready -> In Transit -> Delivered.
FR6	Delivery Job Dispatch & Assignment	Riders can view ready orders, accept delivery tasks, and mark orders picked up and delivered.
FR7	Payment Simulation	Support Cash on Delivery (COD), Credit Card mock gateway, and Digital Wallet (bKash/Nagad simulation).
FR8	Ratings & Reviews	Customers can rate delivered orders (1-5 stars) and provide feedback to update restaurant ratings dynamically.
FR9	Interactive Kanban Board	Real-time visual board displaying order cards across 5 columns with configurable WIP limits and overflow alerts.
FR10	Customer Order History	Customers can review past orders with item breakdown, receipts, and live delivery timeline tracker.
FR11	Admin Analytics & Governance	Administrators can monitor total gross revenue, active WIP count, lead times, and manage all accounts.
FR12	RESTful JSON API	Standardized REST endpoints for restaurants, menus, order status, and Kanban system metrics.

6. Non-Functional Requirements (NFR)

ID	Quality Attribute	Specification Target
NFR1	Performance & Latency	API response time < 150ms for local queries; page load under 1.2s.
NFR2	Data Integrity & Transactions	ACID compliant relational database (SQLite3) with foreign keys and index optimization.
NFR3	Security & Encryption	Passwords hashed with salt using Werkzeug/bcrypt; session cookie protection against CSRF.
NFR4	Usability & Accessibility	Clean UI with Tailwind CSS, high contrast text, responsive grid for desktop and mobile.
NFR5	Maintainability & Modularity	Layered MVC architecture separating models, business services, routes, and views.
NFR6	Testability & Quality Assurance	100% test pass rate across unit and integration tests using pytest.
NFR7	Availability & Fault Tolerance	Graceful exception handling with flash feedback; zero unhandled server crashes.
NFR8	Extensibility	Pluggable architecture allowing future integration of real GPS maps and payment gateways.

7. User Stories & Acceptance Criteria

US1 (Customer Ordering): As a Customer, I want to browse restaurants, add food items to my cart, apply promo codes, and place an order with my delivery address so that I can have food delivered to my room.

Acceptance: Cart calculates subtotal, discounts, and delivery fee; order appears in customer history with 'Placed' status.

US2 (Kitchen Management): As a Restaurant Owner, I want to view incoming orders and transition them to 'Kitchen Preparing' and 'Ready for Pickup' so that food preparation is smooth.

Acceptance: Order updates reflect on the Kanban board; kitchen orders list displays customer special notes.

US3 (Rider Logistics): As a Delivery Rider, I want to view ready orders, accept delivery tasks, and mark them 'In Transit' and 'Delivered' so that I can earn delivery fees.

Acceptance: Rider dashboard shows available and active jobs; total earnings counter increments upon delivery completion.

US4 (Kanban Flow Optimization): As a Kitchen Manager, I want a visual Kanban board with WIP limits so that we prevent kitchen congestion and ensure fast order turnover.

Acceptance: Warning indicator triggers when active cooking tasks exceed the configured WIP limit.

US5 (Platform Governance): As an Administrator, I want to view platform-wide analytics (revenue, throughput, lead times) and oversee registered users.

Acceptance: Real-time KPIs display accurate financial totals and average fulfillment lead time in minutes.

8. Software Process Model Selection: Kanban

For the Food Delivery Application, **Kanban** was selected as the optimal Software Process Model. Kanban is an Agile framework rooted in Lean manufacturing principles that focuses on continuous workflow visualization, Work-In-Progress (WIP) limitation, and cycle time minimization.

Core Justifications for Kanban:

- Direct Domain Symmetry:** The lifecycle of a food order (Placed -> Cooking -> Packaged -> On the Way -> Delivered) is a continuous pull system identical to a Kanban production line.
- WIP Limitation Prevents Bottlenecks:** Enforcing strict WIP limits prevents kitchen queues from ballooning, ensuring food quality and rapid dispatch.
- Continuous Delivery over Fixed Sprints:** Unlike Scrum's rigid 2-week timeboxes, Kanban accommodates urgent production bug fixes and real-time operational enhancements continuously.
- Lead Time & Cycle Time Optimization:** Kanban prioritizes throughput metrics, directly translating to shorter food delivery wait times for customers.

9. Comparison with Alternative Process Models

Process Model	Characteristics	Why It Is Less Suitable for FoodFlow
Waterfall	Linear sequential phases, rigid upfront specifications, late testing phase.	Incapable of adapting to dynamic customer demands, changing payment gateways, or iterative UX feedback without restarting the entire cycle.
Agile Scrum	Fixed time-boxed sprints (1-4 weeks), committed sprint backlog.	Imposes rigid boundaries unsuitable for continuous order dispatching, instant bug patching, and daily operational flow.
Spiral Model	Heavy risk analysis, expensive cyclical prototyping phases.	Introduces extreme management overhead and delays unsuited for an agile consumer-facing web MVP.
Prototype Model	Quick throwaway prototype built before actual development.	Lacks structured process controls for long-term maintainability and operational flow once deployed.

10. MVP System Architecture & Design Overview

The FoodFlow MVP is architected according to professional Software Engineering principles with strict separation of concerns into distinct layers: Presentation (Jinja2/Tailwind UI), Controller / Routing (Flask Blueprints), Business Logic / Services (Service Layer), and Persistence (SQLite3 Relational Schema).

Layer	Module / Component	Responsibilities
Presentation Layer	HTML5, Tailwind CSS, Jinja2 Templates, JavaScript	Customer ordering interface, interactive Kanban board, kitchen dashboard, rider hub, admin control panel.
Routing & Controllers	Flask Blueprints (auth, customer, restaurant, delivery, admin, kanban, api)	Request validation, HTTP status codes, session management, RBAC authorization decorators.
Service Layer	AuthService, RestaurantService, OrderService, KanbanService	Business rule enforcement, state transition validation, WIP limit calculation, promo codes, metrics aggregation.
Persistence Layer	SQLite3 Schema, db.py, seeder.py	Transactional relational tables (users, restaurants, menu_items, orders, order_items, reviews, kanban_tasks).

11. Verification & Quality Assurance

A rigorous automated test suite was constructed using **pytest**, verifying 100% of core system requirements across 20 distinct unit and integration test cases. Test fixtures dynamically provision an isolated temporary SQLite database, verifying user authentication, role enforcement, menu CRUD, cart checkout calculation, state machine integrity, and REST APIs.

12. Challenges Encountered & Mitigations

• **Challenge 1: Managing Kitchen WIP Limits:** During surge order periods, preventing kitchen staff overload without dropping orders.

Mitigation: Implemented a Backlog stage that queues orders safely and alerts managers when cooking WIP threshold is reached.

• **Challenge 2: Invalid Order State Transitions:** Preventing riders or users from skipping fulfillment steps.

Mitigation: Built a formal state transition map in the domain model (`ORDER_STATUS_FLOW`) that strictly validates all transitions.

• **Challenge 3: Role-Based Access Control:** Ensuring customers cannot manipulate kitchen orders or admin settings.

Mitigation: Implemented `@role_required` decorators on Flask routes with role hierarchy verification.

13. Conclusion & Next Milestones

Lab 2 has established a robust, modular, and fully functional Minimum Viable Product for the Food Delivery Application under the Kanban Agile process model. The clean layered architecture and comprehensive automated test suite provide an extensible foundation for upcoming semester labs covering advanced design patterns, continuous integration, and performance profiling.